

Unit-3 MySQL

GTU Syllabus:

Introduction to SQL:

Overview of SQL Query Language, Basic structure of SQL Queries, DDL, DML, Additional Basic Operations, defining constraints – Primary key, foreign key, unique, not null, check, Set operations, Null values, Aggregate functions, Built-in functions –numeric, date, string functions, nested sub- queries, Modification of the Database, DCL and TCL.

What is SQL?

SQL (Structured Query Language) is a language to operate databases; it includes Database Creation, Database Deletion, Fetching Data Rows, Modifying & Deleting Data rows, etc.

SQL stands for Structured Query Language which is a computer language for storing, manipulating and retrieving data stored in a relational database. SQL was developed in the 1970s by IBM Computer Scientists and became a standard of the American National Standards Institute (ANSI) in 1986, and the International Organization for Standardization (ISO) in 1987.

SQL is the standard language to communicate with Relational Database Systems. All the Relational Database Management Systems (RDMS) like MySQL, MS Access, Oracle, Sybase, Informix, Postgres and SQL Server use SQL as their Standard Database Language.

Why SQL?

SQL is widely popular because it offers the following advantages –

- Allows users to access data in the relational database management systems.
- Allows users to describe the data.
- Allows users to define the data in a database and manipulate that data.
- Allows to embed within other languages using SQL modules, libraries & pre-compilers.
- Allows users to create and drop databases and tables.
- Allows users to create view, stored procedure, functions in a database.
- Allows users to set permissions on tables, procedures and views.

A Brief History of SQL

1970 – Dr. Edgar F. "Ted" Codd of IBM is known as the father of relational databases. He described a relational model for databases.

1974 – Structured Query Language (SQL) appeared.

1978 – IBM worked to develop Codd's ideas and released a product named System/R.

1986 – IBM developed the first prototype of relational database and standardized by ANSI. The first relational database was released by Relational Software which later came to be known as Oracle.

1987 – SQL became the part of the International Organization for Standardization (ISO).

SQL Query Structure

SQL is the standard language for managing and querying data in a **Relational Database Management System** (RDBMS). RDBMSs store data in a structured, tabular format with predefined schemas, where tables are linked to one another via relationships.

A basic SQL query, often called a **SELECT** statement, retrieves data from a database. The fundamental structure is:

`SELECT column1, column2 FROM table_name WHERE condition;`

- **SELECT:** Specifies the columns you want to retrieve. Use an asterisk * to select all columns.
- **FROM:** Indicates the table from which to retrieve data.
- **WHERE:** (Optional) Filters the rows based on a specific condition.

Within the SQL language, commands are grouped into five main types:

Data Definition Language (DDL)

DDL commands are used to define or change the structure of a database, like tables. They don't affect the data itself, but rather the containers that hold the data. Think of it as building or modifying the blueprint of a building.

- **CREATE:** Creates a new database or table.
 - Example: `CREATE TABLE Students (StudentID INT, Name VARCHAR(50), City VARCHAR(50));`
- **ALTER:** Modifies an existing table structure, such as adding, deleting, or changing a column.
 - Example: `ALTER TABLE Students ADD Email VARCHAR(100);` to add an 'Email' column.
- **DROP:** Deletes an entire database object. This is a permanent action that removes both the structure and the data.
 - Example: `DROP TABLE Students;`
- **TRUNCATE:** Deletes all records from a table while keeping the table structure. It is faster than DELETE for removing all rows.
 - Example: `TRUNCATE TABLE Students;`

Data Manipulation Language (DML)

DML commands are used to manipulate the data within a database. These are the commands you use most often to interact with the actual information stored in your tables.

- **INSERT:** Adds new rows of data to a table.
 - Example: `INSERT INTO Students (StudentID, Name, City) VALUES (1, 'John Doe', 'New York');`

- **UPDATE:** Modifies existing data in a table.
 - Example: UPDATE Students SET City = 'Los Angeles' WHERE StudentID = 1;
- **DELETE:** Removes one or more rows from a table.
 - Example: DELETE FROM Students WHERE StudentID = 1;

Data Control Language (DCL)

DCL commands manage user access and permissions for a database. They control who can do what with the data and structure.

- **GRANT:** Gives a user specific privileges.
 - Example: GRANT SELECT, INSERT ON Students TO 'user1'@'localhost';
This gives 'user1' permission to view and add data to the Students table.
- **REVOKE:** Removes previously granted privileges from a user.
 - Example: REVOKE INSERT ON Students FROM 'user1'@'localhost'; This removes 'user1's permission to add new data.

Data Query Language (DQL)

DQL is a subset of DML and is used specifically for retrieving data from a database. The **SELECT** statement is the only command in DQL.

- **SELECT:** Retrieves data from one or more tables based on specified conditions.
 - Example: SELECT Name, City FROM Students WHERE City = 'New York';

Transaction Control Language (TCL)

TCL commands manage database transactions. A transaction is a sequence of one or more SQL statements that are executed as a single, indivisible unit of work. TCL commands ensure data integrity and consistency.

- **COMMIT:** Saves all changes made during the current transaction permanently.
 - Example: COMMIT;
- **ROLLBACK:** Undoes all changes since the last COMMIT. This is useful for reverting changes if an error occurs during a transaction.
 - Example: ROLLBACK;
- **SAVEPOINT:** Creates a temporary marker within a transaction to which you can later roll back, without rolling back the entire transaction.
 - Example: SAVEPOINT my_savepoint;

MySQL Data Types (Version 8.0)

Each column in a database table is required to have a name and a data type.

An SQL developer must decide what type of data that will be stored inside each column when creating a table. The data type is a guideline for SQL to understand what type of data is expected inside of each column, and it also identifies how SQL will interact with the stored data.

In MySQL there are three main data types: string, numeric, and date and time.

String Data Types

Data type	Description
CHAR(size)	A FIXED length string (can contain letters, numbers, and special characters). The <i>size</i> parameter specifies the column length in characters - can be from 0 to 255. Default is 1
VARCHAR(size)	A VARIABLE length string (can contain letters, numbers, and special characters). The <i>size</i> parameter specifies the maximum column length in characters - can be from 0 to 65535
BINARY(size)	Equal to CHAR(), but stores binary byte strings. The <i>size</i> parameter specifies the column length in bytes. Default is 1
VARBINARY(size)	Equal to VARCHAR(), but stores binary byte strings. The <i>size</i> parameter specifies the maximum column length in bytes.
TINYBLOB	For BLOBs (Binary Large Objects). Max length: 255 bytes
TINYTEXT	Holds a string with a maximum length of 255 characters
TEXT(size)	Holds a string with a maximum length of 65,535 bytes
BLOB(size)	For BLOBs (Binary Large Objects). Holds up to 65,535 bytes of data

MEDIUMTEXT	Holds a string with a maximum length of 16,777,215 characters
MEDIUMBLOB	For BLOBs (Binary Large Objects). Holds up to 16,777,215 bytes of data
LONGTEXT	Holds a string with a maximum length of 4,294,967,295 characters
LOBLOB	For BLOBs (Binary Large Objects). Holds up to 4,294,967,295 bytes of data
ENUM(val1, val2, val3, ...)	A string object that can have only one value, chosen from a list of possible values. You can list up to 65535 values in an ENUM list. If a value is inserted that is not in the list, a blank value will be inserted. The values are sorted in the order you enter them
SET(val1, val2, val3, ...)	A string object that can have 0 or more values, chosen from a list of possible values. You can list up to 64 values in a SET list

Numeric Data Types

Data type	Description
BIT(<i>size</i>)	A bit-value type. The number of bits per value is specified in <i>size</i> . The <i>size</i> parameter can hold a value from 1 to 64. The default value for <i>size</i> is 1.
TINYINT(<i>size</i>)	A very small integer. Signed range is from -128 to 127. Unsigned range is from 0 to 255. The <i>size</i> parameter specifies the maximum display width (which is 255)
BOOL	Zero is considered as false, nonzero values are considered as true.

BOOLEAN	Equal to BOOL
SMALLINT(<i>size</i>)	A small integer. Signed range is from -32768 to 32767. Unsigned range is from 0 to 65535. The <i>size</i> parameter specifies the maximum display width (which is 255)
MEDIUMINT(<i>size</i>)	A medium integer. Signed range is from -8388608 to 8388607. Unsigned range is from 0 to 16777215. The <i>size</i> parameter specifies the maximum display width (which is 255)
INT(<i>size</i>)	A medium integer. Signed range is from -2147483648 to 2147483647. Unsigned range is from 0 to 4294967295. The <i>size</i> parameter specifies the maximum display width (which is 255)
INTEGER(<i>size</i>)	Equal to INT(<i>size</i>)
BIGINT(<i>size</i>)	A large integer. Signed range is from -9223372036854775808 to 9223372036854775807. Unsigned range is from 0 to 18446744073709551615. The <i>size</i> parameter specifies the maximum display width (which is 255)
FLOAT(<i>size</i> , <i>d</i>)	A floating point number. The total number of digits is specified in <i>size</i> . The number of digits after the decimal point is specified in the <i>d</i> parameter. This syntax is deprecated in MySQL 8.0.17, and it will be removed in future MySQL versions
FLOAT(<i>p</i>)	A floating point number. MySQL uses the <i>p</i> value to determine whether to use FLOAT or DOUBLE for the resulting data type. If <i>p</i> is from 0 to 24, the data type becomes FLOAT(). If <i>p</i> is from 25 to 53, the data type becomes DOUBLE()
DOUBLE(<i>size</i> , <i>d</i>)	A normal-size floating point number. The total number of digits is specified in <i>size</i> . The number of digits after the decimal point is specified in the <i>d</i> parameter

DOUBLE PRECISION(<i>size</i> , <i>d</i>)	
DECIMAL(<i>size</i> , <i>d</i>)	An exact fixed-point number. The total number of digits is specified in <i>size</i> . The number of digits after the decimal point is specified in the <i>d</i> parameter. The maximum number for <i>size</i> is 65. The maximum number for <i>d</i> is 30. The default value for <i>size</i> is 10. The default value for <i>d</i> is 0.
DEC(<i>size</i> , <i>d</i>)	Equal to DECIMAL(<i>size</i> , <i>d</i>)

Note: All the numeric data types may have an extra option: UNSIGNED or ZEROFILL. If you add the UNSIGNED option, MySQL disallows negative values for the column. If you add the ZEROFILL option, MySQL automatically also adds the UNSIGNED attribute to the column.

Date and Time Data Types

Data type	Description
DATE	A date. Format: YYYY-MM-DD. The supported range is from '1000-01-01' to '9999-12-31'
DATETIME(<i>fsp</i>)	A date and time combination. Format: YYYY-MM-DD hh:mm:ss. The supported range is from '1000-01-01 00:00:00' to '9999-12-31 23:59:59'. Adding DEFAULT and ON UPDATE in the column definition to get automatic initialization and updating to the current date and time
TIMESTAMP(<i>fsp</i>)	A timestamp. TIMESTAMP values are stored as the number of seconds since the Unix epoch ('1970-01-01 00:00:00' UTC). Format: YYYY-MM-DD hh:mm:ss. The supported range is from '1970-01-01 00:00:01' UTC to '2038-01-09 03:14:07' UTC. Automatic initialization and updating to the current date and time can be specified using DEFAULT CURRENT_TIMESTAMP and ON UPDATE CURRENT_TIMESTAMP in the column definition
TIME(<i>fsp</i>)	A time. Format: hh:mm:ss. The supported range is from '-838:59:59' to '838:59:59'

YEAR	A year in four-digit format. Values allowed in four-digit format: 1901 to 2155, and 0000. MySQL 8.0 does not support year in two-digit format.
------	--

SQL ALTER TABLE Statement

The **ALTER TABLE** statement is used to add, delete, or modify columns in an existing table.

The **ALTER TABLE** statement is also used to add and drop various constraints on an existing table.

ALTER TABLE - ADD Column

To add a column in a table, use the following syntax:

ALTER TABLE *table_name* **ADD** *column_name* *datatype*;

The following SQL adds an "Email" column to the "Customers" table:

Example :

ALTER TABLE Customers **ADD** Email varchar(255);

ALTER TABLE - DROP COLUMN

To delete a column in a table, use the following syntax (notice that some database systems don't allow deleting a column):

ALTER TABLE *table_name* **DROP COLUMN** *column_name*;

The following SQL deletes the "Email" column from the "Customers" table:

Example:

ALTER TABLE Customers **DROP COLUMN** Email;

ALTER TABLE - RENAME COLUMN

To rename a column in a table, use the following syntax:

ALTER TABLE *table_name* RENAME **COLUMN** *old_name* **to** *new_name*;

To rename a column in a table in SQL Server, use the following syntax:

ALTER TABLE - ALTER/MODIFY DATATYPE

To change the data type of a column in a table, use the following syntax:

My SQL / Oracle (prior version 10G):

ALTER TABLE *table_name* **MODIFY COLUMN** *column_name* *datatype*;

SQL ALTER TABLE Example

The "Persons" table:

ID	LastName	FirstName	Address	City
1	Hansen	Ola	Timoteivn 10	Sandnes
2	Svendson	Tove	Borgvn 23	Sandnes
3	Pettersen	Kari	Storgt 20	Stavanger

Now we want to add a column named "DateOfBirth" in the "Persons" table.

We use the following SQL statement:

ALTER TABLE Persons **ADD** DateOfBirth date;

The "Persons" table will now look like this:

ID	LastName	FirstName	Address	City	DateOfBirth
1	Hansen	Ola	Timoteivn 10	Sandnes	
2	Svendson	Tove	Borgvn 23	Sandnes	
3	Pettersen	Kari	Storgt 20	Stavanger	

Change Data Type Example

Now we want to change the data type of the column named "DateOfBirth" in the "Persons" table.

We use the following SQL statement:

```
ALTER TABLE Persons ALTER COLUMN DateOfBirth year;
```

Notice that the "DateOfBirth" column is now of type year and is going to hold a year in a two- or four-digit format.

DROP COLUMN Example

Next, we want to delete the column named "DateOfBirth" in the "Persons" table.

We use the following SQL statement:

```
ALTER TABLE Persons DROP COLUMN DateOfBirth;
```

The "Persons" table will now look like this:

ID	LastName	FirstName	Address	City
1	Hansen	Ola	Timoteivn 10	Sandnes

2	Svendson	Tove	Borgvn 23	Sandnes
3	Pettersen	Kari	Storgt 20	Stavanger

SQL Constraints:

SQL constraints are used to specify rules for the data in a table.

Constraints are used to limit the type of data that can go into a table. This ensures the accuracy and reliability of the data in the table. If there is any violation between the constraint and the data action, the action is aborted.

Constraints can be column level or table level. Column level constraints apply to a column, and table level constraints apply to the whole table.

The following constraints are commonly used in SQL:

- **NOT NULL** - Ensures that a column cannot have a NULL value
- **UNIQUE** - Ensures that all values in a column are different
- **PRIMARY KEY** - A combination of a **NOT NULL** and **UNIQUE**. Uniquely identifies each row in a table
- **FOREIGN KEY** - Prevents actions that would destroy links between tables
- **CHECK** - Ensures that the values in a column satisfies a specific condition
- **DEFAULT** - Sets a default value for a column if no value is specified
- **CREATE INDEX** - Used to create and retrieve data from the database very quickly

SQL NOT NULL Constraint

By default, a column can hold NULL values.

The **NOT NULL** constraint enforces a column to NOT accept NULL values.

This enforces a field to always contain a value, which means that you cannot insert a new record, or update a record without adding a value to this field.

SQL NOT NULL on CREATE TABLE

The following SQL ensures that the "ID", "LastName", and "FirstName" columns will NOT accept NULL values when the "Persons" table is created:

```
CREATE TABLE Persons (
    ID int NOT NULL,
    LastName varchar(255) NOT NULL,
    FirstName varchar(255) NOT NULL,
    Age int
);
```

SQL UNIQUE Constraint

The **UNIQUE** constraint ensures that all values in a column are different.

Both the **UNIQUE** and **PRIMARY KEY** constraints provide a guarantee for uniqueness for a column or set of columns.

A **PRIMARY KEY** constraint automatically has a **UNIQUE** constraint.

However, you can have many **UNIQUE** constraints per table, but only one **PRIMARY KEY** constraint per table.

SQL UNIQUE Constraint on CREATE TABLE

The following SQL creates a **UNIQUE** constraint on the "ID" column when the "Persons" table is created:

MySQL:

```
CREATE TABLE Persons (
    ID int NOT NULL,
    LastName varchar(255) NOT NULL,
    FirstName varchar(255),
    Age int,
    UNIQUE (ID)
);
```

SQL PRIMARY KEY Constraint

The **PRIMARY KEY** constraint uniquely identifies each record in a table.

Primary keys must contain **UNIQUE** values, and cannot contain **NULL** values.

A table can have only **ONE** primary key; and in the table, this primary key can consist of single or multiple columns (fields).

SQL PRIMARY KEY on CREATE TABLE

The following SQL creates a **PRIMARY KEY** on the "ID" column when the "Persons" table is created:

MySQL:

```
CREATE TABLE Persons (
  ID int NOT NULL,
  LastName varchar(255) NOT NULL,
  FirstName varchar(255),
  Age int,
  PRIMARY KEY (ID)
);
```

SQL FOREIGN KEY Constraint

The **FOREIGN KEY** constraint is used to prevent actions that would destroy links between tables.

A **FOREIGN KEY** is a field (or collection of fields) in one table, that refers to the **PRIMARY KEY** in another table.

The table with the foreign key is called the child table, and the table with the primary key is called the referenced or parent table.

Look at the following two tables:

Persons Table

PersonID	LastName	FirstName	Age
1	Hansen	Ola	30
2	Svendson	Tove	23
3	Pettersen	Kari	20

Orders Table

OrderID	OrderNumber	PersonID
1	77895	3

2	44678	3
3	22456	2
4	24562	1

Notice that the "PersonID" column in the "Orders" table points to the "PersonID" column in the "Persons" table.

The "PersonID" column in the "Persons" table is the **PRIMARY KEY** in the "Persons" table.

The "PersonID" column in the "Orders" table is a **FOREIGN KEY** in the "Orders" table.

The **FOREIGN KEY** constraint prevents invalid data from being inserted into the foreign key column, because it has to be one of the values contained in the parent table.

SQL FOREIGN KEY on CREATE TABLE

The following SQL creates a **FOREIGN KEY** on the "PersonID" column when the "Orders" table is created:

MySQL:

```
CREATE TABLE Orders (
    OrderID int NOT NULL,
    OrderNumber int NOT NULL,
    PersonID int,
    PRIMARY KEY (OrderID),
    FOREIGN KEY (PersonID) REFERENCES Persons(PersonID)
);
```

SQL CHECK Constraint

The **CHECK** constraint is used to limit the value range that can be placed in a column.

If you define a **CHECK** constraint on a column it will allow only certain values for this column.

If you define a **CHECK** constraint on a table it can limit the values in certain columns based on values in other columns in the row.

SQL CHECK on CREATE TABLE

The following SQL creates a **CHECK** constraint on the "Age" column when the "Persons" table is created. The **CHECK** constraint ensures that the age of a person must be 18, or older:

MySQL:

```
CREATE TABLE Persons (
    ID int NOT NULL,
    LastName varchar(255) NOT NULL,
    FirstName varchar(255),
    Age int,
    CHECK (Age>=18)
);
```

SQL DEFAULT Constraint

The **DEFAULT** constraint is used to set a default value for a column.

The default value will be added to all new records, if no other value is specified.

SQL DEFAULT on CREATE TABLE

The following SQL sets a **DEFAULT** value for the "City" column when the "Persons" table is created:

My SQL:

```
CREATE TABLE Persons (
    ID int NOT NULL,
    LastName varchar(255) NOT NULL,
    FirstName varchar(255),
    Age int,
    City varchar(255) DEFAULT 'Sandnes'
);
```

SQL CREATE INDEX Statement

The **CREATE INDEX** statement is used to create indexes in tables.

Indexes are used to retrieve data from the database more quickly than otherwise. The users cannot see the indexes, they are just used to speed up searches/queries.

Note: Updating a table with indexes takes more time than updating a table without (because the indexes also need an update). So, only create indexes on columns that will be frequently searched against.

CREATE INDEX Syntax

Creates an index on a table. Duplicate values are allowed:

```
CREATE INDEX index_name  
ON table_name (column1, column2, ...);
```

CREATE UNIQUE INDEX Syntax

Creates a unique index on a table. Duplicate values are not allowed:

```
CREATE UNIQUE INDEX index_name  
ON table_name (column1, column2, ...);
```

Note: The syntax for creating indexes varies among different databases. Therefore: Check the syntax for creating indexes in your database.

CREATE INDEX Example

The SQL statement below creates an index named "idx_lastname" on the "LastName" column in the "Persons" table:

```
CREATE INDEX idx_lastname  
ON Persons (LastName);
```

If you want to create an index on a combination of columns, you can list the column names within the parentheses, separated by commas:

```
CREATE INDEX idx_pname  
ON Persons (LastName, FirstName);
```

Example of SQL Queries:

Step 1: Create Sample Table & Insert Data

```
CREATE TABLE Employees (  
    EmpID INT PRIMARY KEY,  
    Name VARCHAR(50),  
    Age INT,  
    Department VARCHAR(50),  
    Salary DECIMAL(10,2),  
    City VARCHAR(50),  
    Commission DECIMAL(10,2)  
);
```

```
INSERT INTO Employees VALUES
```

(101, 'Alice', 30, 'HR', 50000.00, 'Mumbai', NULL),
(102, 'Bob', 25, 'IT', 60000.00, 'Delhi', 3000.00),
(103, 'Charlie', 28, 'Finance', 55000.00, NULL, NULL),
(104, 'David', 35, 'IT', 75000.00, 'Chennai', 5000.00),
(105, 'Eva', 40, 'HR', 70000.00, 'Mumbai', 4500.00),
(106, 'Frank', 23, 'Finance', 48000.00, 'Pune', NULL);

Step 2: Practice Queries

1. SELECT Queries

1. Display all records

```
SELECT * FROM Employees;
```

2. Display names and salaries

```
SELECT Name, Salary FROM Employees;
```

3. Display Employee ID and Department

```
SELECT EmpID, Department FROM Employees;
```

2. WHERE Clause

4. Employees from IT department

```
SELECT * FROM Employees WHERE Department = 'IT';
```

5. Employees older than 30

```
SELECT * FROM Employees WHERE Age > 30;
```

6. Employees with salary less than or equal to 60000

```
SELECT * FROM Employees WHERE Salary <= 60000;
```

3. ORDER BY Clause

7. Employees sorted by Salary (descending)

```
SELECT * FROM Employees ORDER BY Salary DESC;
```

8. Employees sorted by Age (ascending)

```
SELECT * FROM Employees ORDER BY Age ASC;
```

9. Sort by Department, then by Name

```
SELECT * FROM Employees ORDER BY Department, Name;
```

4. IN Operator:

The IN operator allows you to specify multiple values in a WHERE clause.

The IN operator is a shorthand for multiple OR conditions.

10. Employees from HR or Finance

```
SELECT * FROM Employees WHERE Department IN ('HR', 'Finance');
```

11. Employees in Mumbai or Delhi

```
SELECT * FROM Employees WHERE City IN ('Mumbai', 'Delhi');
```

NOT IN:

By using the NOT keyword in front of the IN operator, you return all records that are NOT any of the values in the list.

Example:

Employees not in Mumbai or Delhi

```
SELECT * FROM Employees WHERE City NOT IN ('Mumbai', 'Delhi');
```

5. BETWEEN Operator:

The SQL BETWEEN Operator

The **BETWEEN** operator selects values within a given range. The values can be numbers, text, or dates.

The **BETWEEN** operator is inclusive: begin and end values are included.

12. Employees earning between 50000 and 70000

```
SELECT * FROM Employees WHERE Salary BETWEEN 50000 AND 70000;
```

13. Employees aged between 25 and 35

```
SELECT * FROM Employees WHERE Age BETWEEN 25 AND 35;
```

6. IS NULL and IS NOT NULL

14. Employees without commission

```
SELECT * FROM Employees WHERE Commission IS NULL;
```

15. Employees with commission

```
SELECT * FROM Employees WHERE Commission IS NOT NULL;
```

16. Employees whose city is unknown

```
SELECT * FROM Employees WHERE City IS NULL;
```

7. LIKE Operator:

The SQL LIKE Operator

The **LIKE** operator is used in a **WHERE** clause to search for a specified pattern in a column.

There are two wildcards often used in conjunction with the **LIKE** operator:

- The percent sign **%** represents zero, one, or multiple characters
- The underscore sign **_** represents one, single character

17. Names starting with 'A'

```
SELECT * FROM Employees WHERE Name LIKE 'A%';
```

18. Names ending with 'a'

```
SELECT * FROM Employees WHERE Name LIKE '%a';
```

19. Names containing 'a'

```
SELECT * FROM Employees WHERE Name LIKE '%a%';
```

20. Names where second letter is 'a'

```
SELECT * FROM Employees WHERE Name LIKE '_a%';
```

Combination Queries

21. IT department employees with salary > 60000

```
SELECT * FROM Employees WHERE Department = 'IT' AND Salary > 60000;
```

22. Mumbai or Pune employees without commission

```
SELECT * FROM Employees WHERE City IN ('Mumbai', 'Pune') AND Commission IS NULL;
```

-- 23. Names starting with D, sorted by Age

```
SELECT * FROM Employees WHERE Name LIKE 'D%' ORDER BY Age DESC;
```

-- 24. HR employees earning between 50000 and 70000

```
SELECT * FROM Employees WHERE Department = 'HR' AND Salary BETWEEN 50000 AND 70000;
```

-- 25. Employees with commission, sorted by commission

```
SELECT * FROM Employees WHERE Commission IS NOT NULL ORDER BY Commission DESC;
```

Assume we are working with this table:

Table Name: Employees

1. UPDATE Queries

Modify data in existing records

1. Increase salary by 5000 for employees in IT

```
UPDATE Employees SET Salary = Salary + 5000
WHERE Department = 'IT';
```

2. Set Commission to 1000 where it is NULL

```
UPDATE Employees SET Commission = 1000 WHERE Commission IS NULL;
```

3. Change City of employee 'Charlie' to 'Bangalore'

```
UPDATE Employees SET City = 'Bangalore' WHERE Name = 'Charlie';
```

4. Reduce salary by 10% for employees in Finance

```
UPDATE Employees
SET Salary = Salary * 0.9
WHERE Department = 'Finance';
```

5. Set Department to NULL for EmpID = 105

```
UPDATE Employees SET Department = NULL WHERE EmpID = 105;
```

2. DELETE Queries

Delete rows from table

6. Delete employee with EmpID 106

```
DELETE FROM Employees
WHERE EmpID = 106;
```

7. Delete all employees from HR department

```
DELETE FROM Employees
WHERE Department = 'HR';
```

8. Delete all employees with no commission

```
DELETE FROM Employees WHERE Commission IS NULL;
```

-- 9. Delete employees earning less than 50000

```
DELETE FROM Employees WHERE Salary < 50000;
```

10. Delete all employees whose city is NULL

```
DELETE FROM Employees WHERE City IS NULL;
```

3. DROP TABLE

□ Completely remove table and structure

11. Drop the Employees table (irreversible!)

```
DROP TABLE Employees;
```

4. TRUNCATE TABLE

□ **Delete all records but keep table structure**

12. Remove all data but keep table structure

TRUNCATE TABLE Employees;

5. ALTER TABLE Queries

Modify structure (add, drop, change columns)

13. Add a new column 'JoinDate'

ALTER TABLE Employees ADD JoinDate DATE;

14. Drop the column 'Commission'

ALTER TABLE Employees DROP COLUMN Commission;

15. Rename column 'Name' to 'FullName'

ALTER TABLE Employees RENAME COLUMN Name TO FullName;

16. Change data type of Salary to BIGINT

ALTER TABLE Employees MODIFY Salary BIGINT;

17. Rename the table to 'EmployeeDetails'

ALTER TABLE Employees RENAME TO EmployeeDetails;

The SQL GROUP BY Statement

The **GROUP BY** statement groups rows that have the same values into summary rows, like "find the number of customers in each country".

The **GROUP BY** statement is often used with aggregate functions (**COUNT()**, **MAX()**, **MIN()**, **SUM()**, **AVG()**) to group the result-set by one or more columns.

What is GROUP BY?

The GROUP BY clause groups rows that have the **same values in specified columns** into **summary rows**, like:

- Total salary by department
- Average age by city
- Count of employees in each department

It's commonly used with **aggregate functions** like SUM(), COUNT(), AVG(), MIN(), MAX().

1. Total Salary by Department

```
SELECT Department, SUM(Salary) AS TotalSalary
FROM Employees
GROUP BY Department;
```

2. Average Age by Department

```
SELECT Department, AVG(Age) AS AverageAge
FROM Employees
GROUP BY Department;
```

3. Count of Employees in Each City

```
SELECT City, COUNT(*) AS TotalEmployees
FROM Employees
GROUP BY City;
```

4. Minimum and Maximum Salary by Department

```
SELECT Department, MIN(Salary) AS MinSalary, MAX(Salary) AS MaxSalary
FROM Employees
GROUP BY Department;
```

What Are Aggregate Functions?

Aggregate functions perform a **calculation on a set of values** and return a **single value**. They are commonly used with **GROUP BY**, **HAVING**, and **WHERE** clauses.

Common Aggregate Functions

Function	Description
COUNT()	Counts number of rows
SUM()	Calculates total sum of a column
AVG()	Calculates average of a numeric column
MIN()	Finds minimum value
MAX()	Finds maximum value

Sample Table: Employees

```
CREATE TABLE Employees (
    EmpID INT,
    Name VARCHAR(50),
    Department VARCHAR(50),
    Salary DECIMAL(10,2),
    Age INT,
    City VARCHAR(50)
);
```

```
INSERT INTO Employees VALUES
(101, 'Alice', 'HR', 50000, 30, 'Mumbai'),
(102, 'Bob', 'IT', 60000, 25, 'Delhi'),
(103, 'Charlie', 'Finance', 55000, 28, 'Bangalore'),
```

(104, 'David', 'IT', 75000, 35, 'Chennai'),
(105, 'Eva', 'HR', 70000, 40, 'Mumbai'),
(106, 'Frank', 'Finance', 48000, 23, 'Pune');

Practice Queries:

1. COUNT()

1. Total number of employees

SELECT COUNT(*) FROM Employees;

2. Count employees in IT department

SELECT COUNT(*) FROM Employees WHERE Department = 'IT';

2. SUM()

3. Total salary paid to all employees

SELECT SUM(Salary) AS TotalSalary FROM Employees;

4. Total salary paid to HR department

SELECT SUM(Salary) AS HRTotals FROM Employees WHERE Department = 'HR';

3. AVG()

5. Average salary of all employees

SELECT AVG(Salary) AS AverageSalary FROM Employees;

6. Average age of employees in Finance

SELECT AVG(Age) AS FinanceAvgAge FROM Employees WHERE Department = 'Finance';

4. MIN() and MAX()

7. Minimum salary

SELECT MIN(Salary) AS MinSalary FROM Employees;

8. Maximum salary

SELECT MAX(Salary) AS MaxSalary FROM Employees;

9. Youngest employee age

SELECT MIN(Age) AS Youngest FROM Employees;

10. Oldest employee age

SELECT MAX(Age) AS Oldest FROM Employees;

5. Group-wise Aggregation (GROUP BY)

11. Total salary by department

```
SELECT Department, SUM(Salary) AS DeptTotal FROM Employees GROUP BY
Department;
```

```
12. Count of employees per city
SELECT City, COUNT(*) AS CityWiseCount
FROM Employees
GROUP BY City;
```

Single Row Functions in MySQL

Single Row Functions (also called *scalar functions*) operate on a **single row of data** and return **one result per row**. These are useful for **data formatting, manipulation, or computation** in SQL queries.

Types of Single Row Functions

MySQL provides several categories of single row functions:

Function Type	Examples
String Functions	UPPER(), LOWER(), CONCAT(), SUBSTRING()
Numeric Functions	ROUND(), CEIL(), FLOOR(), MOD()
Date & Time Functions	NOW(), CURDATE(), DATE_FORMAT()
Conversion Functions	CAST(), CONVERT()
Control Functions	IF(), IFNULL(), NULLIF()

1. String Functions

String functions are used to **manipulate and process string/text data**.

Function	Description	Example	Output
UPPER()	Converts string to uppercase	SELECT UPPER('mysql');	MYSQL
LOWER()	Converts string to lowercase	SELECT LOWER('MySQL');	mysql
LENGTH()	Returns length of string in bytes	SELECT LENGTH('abc');	3
CHAR_LENGTH()	Returns number of characters	SELECT CHAR_LENGTH('नमस्ते');	6
CONCAT()	Joins two or more strings	SELECT CONCAT('Hello', 'World');	Hello World
SUBSTRING()	Extracts part of a string	SELECT SUBSTRING('Database', 2, 4);	ATAB
TRIM()	Removes leading and trailing spaces	SELECT TRIM(' MySQL ');	MySQL
REPLACE()	Replaces a part of the string	SELECT REPLACE('black hat', 'black', 'white');	white hat

2. Numeric Functions

These functions are used for **performing operations on numeric values**.

Function	Description	Example	Output
ROUND()	Rounds a number to the nearest integer or given decimal places	SELECT ROUND(123.456, 2);	123.46
CEIL() / CEILING()	Rounds up to the next integer	SELECT CEIL(12.1);	13
FLOOR()	Rounds down to the previous integer	SELECT FLOOR(12.9);	12
MOD()	Returns remainder of division	SELECT MOD(10, 3);	1
ABS()	Returns absolute (positive) value	SELECT ABS(-25);	25
POWER()	Returns value raised to power	SELECT POWER(2, 3);	8

3. Date and Time Functions

Used for handling **date and time values**.

Function	Description	Example	Output
NOW()	Returns current date and time	SELECT NOW();	2025-07-29 11:25:00
CURDATE()	Returns current date	SELECT CURDATE();	2025-07-29
CURTIME()	Returns current time	SELECT CURTIME();	11:25:00
YEAR()	Extracts year from date	SELECT YEAR('2025-07-29');	2025
MONTH()	Extracts month from date	SELECT MONTH('2025-07-29');	7
DAY()	Extracts day from date	SELECT DAY('2025-07-29');	29
DATE_FORMAT()	Formats date/time	SELECT DATE_FORMAT(NOW(), '%d-%m-%Y');	29-07-2025

4. Conversion Functions

Used to **convert data types**.

Function	Description	Example	Output
CAST()	Converts a value from one data type to another	SELECT CAST('123' AS UNSIGNED);	123
CONVERT()	Similar to CAST for type conversion	SELECT CONVERT('2025-07-29', DATE);	2025-07-29

5. Control Functions

Used for **conditional logic and handling NULLs**.

Function	Description	Example	Output
IF()	Returns one value if condition is TRUE, another if FALSE	SELECT IF(5 > 3, 'Yes', 'No');	Yes
IFNULL()	Returns second value if first is NULL	SELECT IFNULL(NULL, 'Default');	Default
NULLIF()	Returns NULL if both arguments are equal	SELECT NULLIF(10, 10);	NULL
	Otherwise returns first argument	SELECT NULLIF(10, 20);	10

What is a Foreign Key?

A **foreign key** is a column (or group of columns) in one table that **references the PRIMARY KEY** in another table. It is used to **enforce referential integrity**, meaning it ensures that the relationship between two tables remains consistent.

Purpose of Foreign Key

- Establish a **relationship** between two tables.
- Ensure **data consistency**.
- Prevent **orphan records** (records that refer to non-existent rows).

Example Scenario:

Let's take a real-world case:

- A **Department** can have **many Employees**.
- Each employee must belong to a valid department.

Step 1: Create the Department table (Parent Table)

```
CREATE TABLE Department (
    dept_id INT PRIMARY KEY,
    dept_name VARCHAR(50)
);
```

Step 2: Create the Employee table (Child Table) with Foreign Key

```
CREATE TABLE Employee (
    emp_id INT PRIMARY KEY,
    emp_name VARCHAR(100),
    dept_id INT,
    FOREIGN KEY (dept_id) REFERENCES Department(dept_id)
);
```

Here:

- dept_id in **Employee** table is a **foreign key**.
- It references dept_id in **Department** table.

□ How it Works:

- You **can only insert** an employee with a dept_id that **already exists** in the Department table.
- If you **try to insert** an employee with a non-existent dept_id, **MySQL will throw an error**.

✓ Example Insertions

Insert into Department (Parent Table):

```
INSERT INTO Department (dept_id, dept_name)
VALUES (101, 'HR'), (102, 'Engineering');
```

Insert into Employee (Child Table):

-- Valid entry: dept_id = 101 exists

```
INSERT INTO Employee (emp_id, emp_name, dept_id) VALUES (1, 'Alice', 101);
```

Invalid entry: dept_id = 999 does NOT exist

```
INSERT INTO Employee (emp_id, emp_name, dept_id)
VALUES (2, 'Bob', 999);
```

Error: foreign key constraint fails

Referential Integrity Rules

- You **can't insert** a child record unless the parent record exists.
- You **can't delete** a parent record if there are related child records (unless you handle it explicitly using ON DELETE rules).

Join in MySQL:

To combine rows from two or more tables in MySQL, you can use the **JOIN** clause. Joins work by finding related columns between the tables and using a join condition to match the rows. The most common type of join is an **INNER JOIN**.

Types of Joins

There are several types of joins in MySQL, each with a different purpose:

- **INNER JOIN**: This is the most common type. It returns only the rows that have matching values in both tables. Any rows without a match in the other table are excluded.

- **LEFT (OUTER) JOIN:** This returns all rows from the **left** table and the matched rows from the right table. If there's no match in the right table, NULL values are returned for the columns from the right table.
- **RIGHT (OUTER) JOIN:** This is the opposite of a LEFT JOIN. It returns all rows from the **right** table and the matched rows from the left table. If there's no match in the left table, NULL values are returned for the columns from the left table.
- **FULL (OUTER) JOIN:** This returns all rows when there is a match in either the left or right table. It combines the results of both LEFT and RIGHT joins. Note that MySQL doesn't directly support FULL OUTER JOIN; you typically achieve this by using a UNION of a LEFT JOIN and a RIGHT JOIN.
- **CROSS JOIN:** This returns the Cartesian product of the two tables. Every row from the first table is combined with every row from the second table. This is rarely used in practice unless you have a specific reason to combine every possible pair of rows.

Syntax

The basic syntax for a JOIN is as follows:

```
SQL
SELECT column_name(s)
FROM table1
JOIN_TYPE table2
ON table1.column_name = table2.column_name;
```

Example

Let's imagine you have two tables: customers and orders. The customers table has customer_id and customer_name, and the orders table has order_id, customer_id, and order_date. To find the order details along with the customer's name, you can use an INNER JOIN.

```
SQL
SELECT c.customer_name, o.order_id, o.order_date
FROM customers c
INNER JOIN orders o
ON c.customer_id = o.customer_id;
```

In this example:

- c and o are aliases for the tables customers and orders, which makes the query easier to read.
- The ON clause specifies the **join condition**, which is the common column (customer_id) that links the two tables.

Practical Example

Let's assume we have these two tables:

```
Create table employees( employee_id int primary key,employee_name
varchar(50),department_id int);
```

```
Create table departments(department_id int primary key,department_name
varchar(50));
```

employees table:

employee_id	employee_name	department_id
1	Alice	101
2	Bob	102
3	Charlie	101
4	David	103

Departments Table:

department_id	department_name
101	Sales
102	Marketing
104	HR

1. INNER JOIN

To get the name of each employee and their corresponding department name, we use an INNER JOIN:

SQL

```
SELECT e.employee_name, d.department_name
FROM employees e
INNER JOIN departments d
ON e.department_id = d.department_id;
```

Result:

employee_name	department_name
Alice	Sales
Bob	Marketing
Charlie	Sales

- David is excluded because his department_id (103) doesn't exist in the departments table.
- HR is excluded because no employees have a department_id of 104.

2. LEFT JOIN

To get all employees, even those without a matching department, we use a LEFT JOIN:

SQL

```
SELECT e.employee_name, d.department_name
FROM employees e
LEFT JOIN departments d
ON e.department_id = d.department_id;
```

Result:

employee_name	department_name
Alice	Sales
Bob	Marketing
Charlie	Sales
David	NULL

- This query returns all employees from the employees table. For David, since there's no match in the departments table, the department_name is NULL.

3. RIGHT JOIN

To get all departments, even those without an employee, we use a RIGHT JOIN:

SQL

```
SELECT e.employee_name, d.department_name
FROM employees e
RIGHT JOIN departments d
ON e.department_id = d.department_id;
```

Result:

employee_name	department_name
Alice	Sales
Bob	Marketing
Charlie	Sales
NULL	HR

4. CROSS JOIN

A **CROSS JOIN** produces a Cartesian product of the two tables. This means every row from the first table is combined with every row from the second table. This type of join is useful for generating all possible combinations of rows between two tables.

Using your provided tables, a CROSS JOIN would combine each of the four employees with each of the three departments, resulting in 12 rows.

SQL

```

SELECT
e.employee_id,e.employee_name,e.department_id,d.department_name,d.department_id
FROM employees e
CROSS JOIN departments d;

```

Result:

employee_name	department_name
Alice	Sales
Bob	Sales
Charlie	Sales
David	Sales
Alice	Marketing
Bob	Marketing
Charlie	Marketing
David	Marketing
Alice	HR
Bob	HR
Charlie	HR
David	HR

5. FULL (OUTER) JOIN

A **FULL JOIN** returns all rows from both the left and right tables, with NULL values in columns where there is no match. MySQL doesn't have a direct FULL JOIN syntax, but you can achieve the same result by combining a LEFT JOIN and a RIGHT JOIN using the UNION operator. The UNION operator combines the result sets of two or more SELECT statements.

SQL

```

SELECT e.employee_name, d.department_name
FROM employees e
LEFT JOIN departments d ON e.department_id = d.department_id
UNION
SELECT e.employee_name, d.department_name
FROM employees e
RIGHT JOIN departments d ON e.department_id = d.department_id;

```

Result:

employee_name	department_name
Alice	Sales
Bob	Marketing
Charlie	Sales
David	NULL
NULL	HR

6.SELF JOIN

A **SELF JOIN** is a regular join that joins a table to itself. This is useful when you need to compare rows within the same table. It requires aliasing the table to treat it as two separate entities.

Let's assume the employees table has a manager_id column that references the employee_id of another employee in the same table.

Modified employees table:

employee_id	employee_name	department_id	manager_id
1	Alice	101	2
2	Bob	102	3
3	Charlie	101	4
4	David	103	2

To find an employee's name and their manager's name, you would perform a self-join.

SQL

```
SELECT e1.employee_name AS Employee, e2.employee_name AS Manager
FROM employees e1
JOIN employees e2 ON e1.manager_id = e2.employee_id;
```

Result:

Employee	Manager
Bob	Alice
Charlie	Alice
David	Bob

How to use ANY, ALL, EXISTS, and HAVING clauses with your specified MySQL tables, employees and departments.

ANY

The **ANY** clause is used with a subquery and returns true if any of the subquery values meet the condition.

Example: Find employees who work in departments that have an employee_id less than 10.

SQL

```
SELECT employee_name FROM employees
WHERE department_id = ANY (SELECT department_id FROM employees WHERE
employee_id < 10);
```

This query selects all employees whose department_id matches at least one department_id from the subquery's result set.

ALL

The **ALL** clause is used with a subquery and returns true only if all of the subquery values meet the condition.

Example: Find employees whose department_id is less than all department_ids in the departments table.

SQL

```
SELECT employee_name FROM employees  
WHERE department_id < ALL (SELECT department_id FROM departments);
```

This query will return employees only if their department_id is smaller than every single department_id in the departments table.

EXISTS

The **EXISTS** clause checks for the existence of rows in a subquery. It is more efficient than ANY or IN for large datasets.

Example: Find employees who have a corresponding entry in the departments table.

SQL

```
SELECT employee_name FROM employees e  
WHERE EXISTS (SELECT 1 FROM departments d  
WHERE d.department_id = e.department_id);
```

This query checks if a matching department_id exists in the departments table for each employee. The SELECT 1 is a common, efficient practice as it doesn't need to retrieve any specific data, just check for existence.

HAVING

The **HAVING** clause is used to filter groups of rows created by the GROUP BY clause. It cannot be used without a GROUP BY clause.

Example: Find departments with more than one employee.

SQL

```
SELECT d.department_name, COUNT(e.employee_id) AS number_of_employees
FROM employees e
JOIN departments d ON e.department_id = d.department_id
GROUP BY d.department_name
HAVING COUNT(e.employee_id) > 1;
```

Subquery:

A subquery is a query nested inside another query. In the context of your provided tables (employees and departments), here's an example:

Problem: Find the names of employees who work in the 'Sales' department.

Solution using a subquery:

1. First, find the department_id for 'Sales' using an inner query.
2. Then, use that department_id to find the corresponding employee names in the outer query.

SQL

```
SELECT employee_name
FROM employees
WHERE department_id = (SELECT department_id FROM departments WHERE
department_name = 'Sales');
```

Explanation:

- The **inner query** (SELECT department_id FROM departments WHERE department_name = 'Sales') is executed first. It returns the value 101.
- The **outer query** then becomes SELECT employee_name FROM employees WHERE department_id = 101.
- The final result is Alice and Charlie, as they are the employees with department_id 101.

Set operations:

UNION, INTERSECT, and EXCEPT are SQL set operators used to combine the results of two or more SELECT statements. They are often used when you need to perform operations on result sets rather than individual tables.

UNION

The **UNION** operator combines the result sets of two or more SELECT statements into a single result set. It removes duplicate rows by default. To include all rows, including duplicates, you can use **UNION ALL**. The number of columns and their data types must be the same in all SELECT statements within the UNION query.

Example: Find all unique employee and department names.

SQL

```
SELECT employee_name FROM employees  
UNION  
SELECT department_name FROM departments;
```

INTERSECT

The **INTERSECT** operator returns only the rows that are present in both result sets. It's like finding the common elements between two sets. Most database systems, including MySQL, do not support INTERSECT directly. However, you can achieve the same result using a subquery and JOIN.

Example: Find department IDs that have both an entry in the employees table and the departments table.

SQL

```
SELECT department_id FROM employees  
WHERE department_id IN (SELECT department_id FROM departments);
```

This query returns only the department IDs that exist in both tables.

EXCEPT / MINUS

The **EXCEPT** or **MINUS** operator returns the rows from the first SELECT statement that are not present in the second SELECT statement. MySQL does not directly support the EXCEPT or MINUS operator. Similar to INTERSECT, you can simulate this behavior using a LEFT JOIN and checking for NULL values.

Example: Find department IDs that exist in the departments table but have no employees in the employees table.

SQL

```
SELECT d.department_id  
FROM departments d  
LEFT JOIN employees e ON d.department_id = e.department_id  
WHERE e.department_id IS NULL;
```

This query joins departments with employees and returns only the rows where there's no match in the employees table (i.e., e.department_id is NULL).

Transaction Control Language (TCL)

TCL commands manage transactions in a database. A transaction is a single unit of work, which can consist of multiple SQL statements.

COMMIT

The COMMIT command permanently saves a transaction to the database. Once you COMMIT, the changes cannot be undone with ROLLBACK.

Example: Suppose you want to add a new employee and a new department. You wrap these two insertions in a transaction.

SQL

```
START TRANSACTION;  
INSERT INTO employees (employee_id, employee_name, department_id) VALUES (5,  
'Frank', 104);  
INSERT INTO departments (department_id, department_name) VALUES (105, 'Research');  
COMMIT;
```

After executing COMMIT, the changes for both the new employee and the new department are permanent and visible to other users.

ROLLBACK

The ROLLBACK command undoes all changes made in the current transaction since the last COMMIT or START TRANSACTION.

Example: Imagine you try to add an employee, but then realize you made a mistake and want to undo the change before committing.

SQL

```
START TRANSACTION;  
INSERT INTO employees (employee_id, employee_name, department_id) VALUES (6,  
'Grace', 101);  
-- You realize the employee_id should be 7, not 6.  
ROLLBACK;
```

The ROLLBACK command cancels the INSERT, and the table reverts to its state before the START TRANSACTION command. The new employee Grace is not added to the table.

SAVEPOINT

A SAVEPOINT is a marker within a transaction that allows you to roll back to a specific point without rolling back the entire transaction.

Example: You are making several changes but want the ability to undo only the last change.

SQL

```
START TRANSACTION;
INSERT INTO employees (employee_id, employee_name, department_id) VALUES (7,
'Heidi', 102);
SAVEPOINT insert_heidi;
INSERT INTO employees (employee_id, employee_name, department_id) VALUES (8, 'Ivan',
101);
-- You decide you want to keep 'Heidi' but not 'Ivan'.
ROLLBACK TO insert_heidi;
COMMIT;
```

ROLLBACK TO insert_heidi undoes the insertion of Ivan but keeps Heidi. The final COMMIT then saves the changes, including Heidi.

Data Control Language (DCL)

DCL commands manage user privileges and permissions in the database.

GRANT

The GRANT command provides specific privileges to a user.

Example: Suppose you have a user named 'analyst' and you want to give them permission to read data from the employees table.

SQL

```
GRANT SELECT ON employees TO 'analyst'@'localhost';
```

This command gives the analyst user the right to execute SELECT statements on the employees table.

REVOKE

The REVOKE command removes privileges from a user.

Example: If the 'analyst' no longer needs access to the employees data, you can revoke their permissions.

SQL

```
REVOKE SELECT ON employees FROM 'analyst'@'localhost';
```

This command takes away the SELECT privilege from the analyst user, preventing them from querying the employees table.

JOIN Practice example:

Tables:

```
1.CREATE TABLE Students (
```

```
    student_id INT PRIMARY KEY,
```

```
    name VARCHAR(50),
```

```
    department_id INT);
```

```
2.CREATE TABLE Departments (
```

```
    department_id INT PRIMARY KEY,
```

```
    department_name VARCHAR(50)
```

```
);
```

```
3. CREATE TABLE Courses (
```

```
    course_id INT PRIMARY KEY,
```

```
    course_name VARCHAR(50),
```

```
    department_id INT
```

```
);
```

```
CREATE TABLE Enrollments (
```

```
    enrollment_id INT PRIMARY KEY,
```

```
    student_id INT,
```

```
    course_id INT,
```

```
    grade CHAR(2)
```

```
);
```

Practice Query:

1. INNER JOIN with 3 tables:

Get list of students with their department and the course they are enrolled in.

```
SELECT s.name AS student_name,  
       d.department_name,  
       c.course_name FROM Students JOIN Departments d ON s.department_id =  
d.department_id  
JOIN Enrollments e ON s.student_id = e.student_id  
JOIN Courses c ON e.course_id = c.course_id;
```

2. left join: Show all students with their department and course enrolment.

```
SELECT s.name AS student_name,  
       d.department_name,  
       c.course_name  
FROM Students s  
LEFT JOIN Departments d ON s.department_id = d.department_id  
LEFT JOIN Enrollments e ON s.student_id = e.student_id  
LEFT JOIN Courses c ON e.course_id = c.course_id;
```

3. Find number of students enrolled in each course(With department info).

```
SELECT c.course_name,  
       d.department_name,  
       COUNT(e.student_id) AS total_students  
FROM Courses c  
JOIN Departments d ON c.department_id = d.department_id  
LEFT JOIN Enrollments e ON c.course_id = e.course_id  
GROUP BY c.course_name, d.department_name;
```